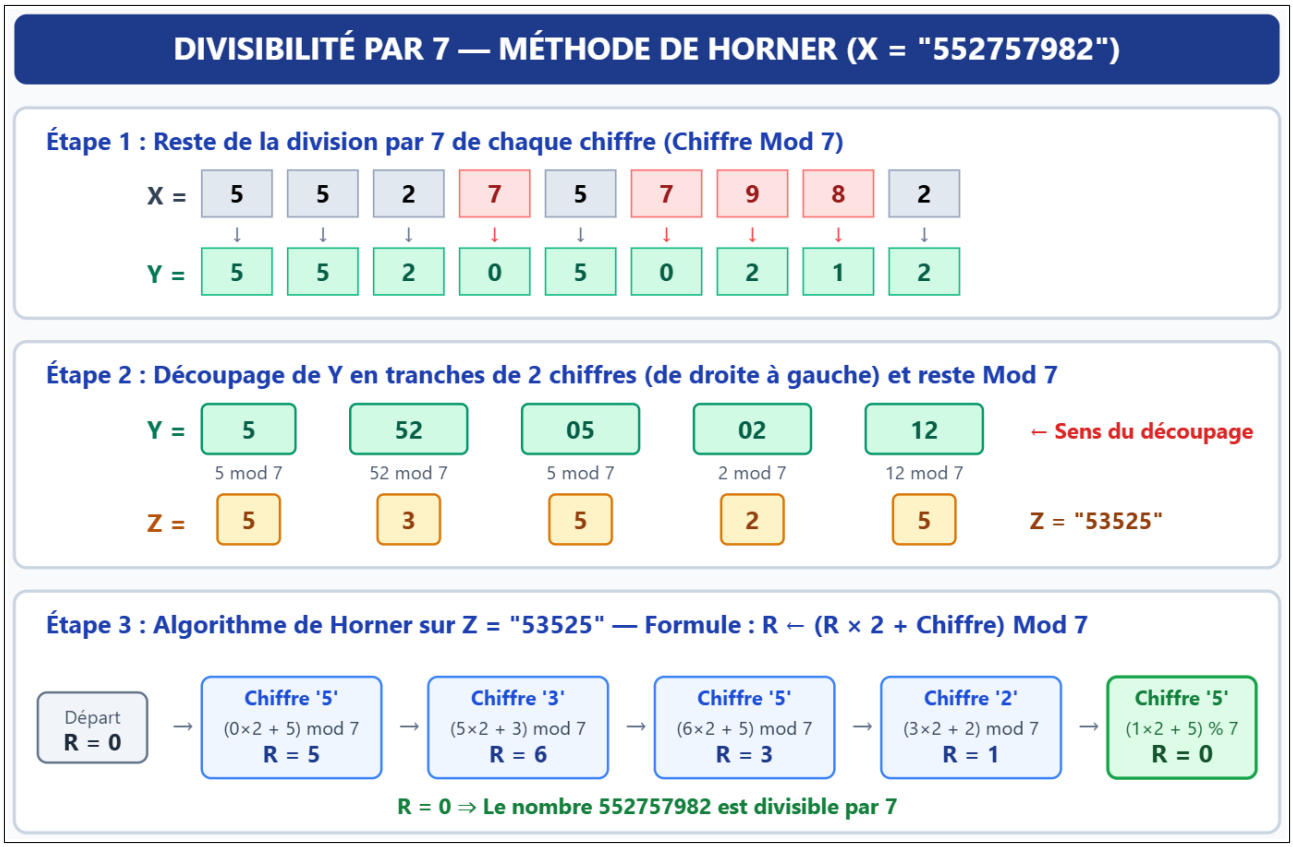
Série d'exercices n°1

Exercice 1 : Divisibilité par 7 (Méthode de Horner)

On souhaite vérifier si un nombre X (chaîne de 5 à 20 chiffres) est divisible par 7 en trois étapes :

1. **Étape 1 :** Former la chaîne Y en remplaçant chaque chiffre de X par le reste de sa division par 7.
2. **Étape 2 :** Découper Y en tranches de 2 chiffres de droite à gauche (la dernière tranche à gauche peut contenir un seul chiffre si la longueur est impaire). Former Z en remplaçant chaque tranche par le reste de sa division par 7.
3. **Étape 3 :** Calculer le reste final R à partir de Z avec l'algorithme de Horner :
   - Initialiser R à 0.
   - Pour chaque chiffre c de Z (de gauche à droite) :

     `R ← (R * 2 + Valeur(c)) Mod 7`.
   - Si `R = 0`, alors X est divisible par 7.



Exemple :

Pour `X = "552757982"` :

☑ **Étape 1 :** `Y = "552050212"` (car `7 mod 7 = 0`, `9 mod 7 = 2`, `8 mod 7 = 1`, etc.).

☑ **Étape 2 :** Découpage de Y : `5 | 52 | 05 | 02 | 12`. Restes modulo 7 : `5 mod 7 = 5` ; `52 mod 7 = 3` ; `05 mod 7 = 5` ; `02 mod 7 = 2` ; `12 mod 7 = 5` → `Z = "53525"`.

☑ **Étape 3 :** Application de Horner sur `Z = "53525"` → `R = 0`. Donc `552757982` est divisible par 7.

Travail demandé :

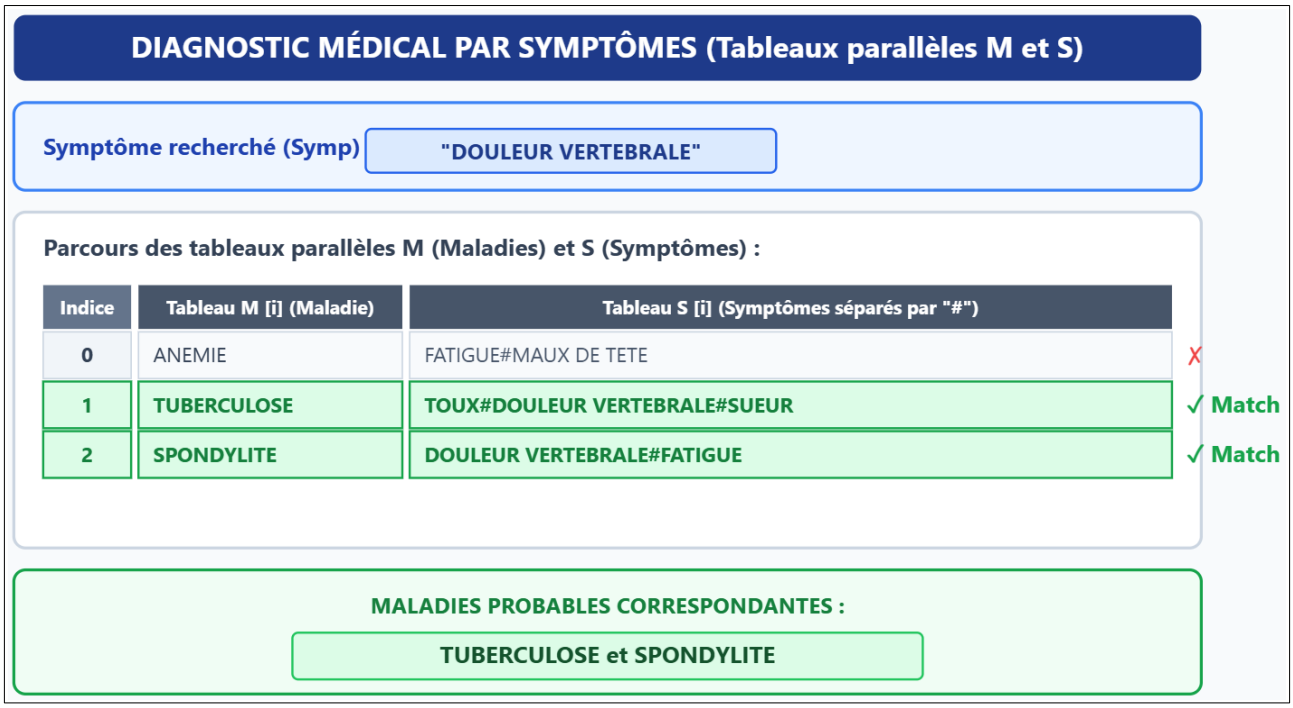
Écrire l'algorithme du programme principal et des modules permettant de saisir `X` et d'afficher s'il est divisible par 7.

Exercice 2 : Diagnostic médical

On dispose de `N` maladies ($2 < N \leq 100$) :

- Un tableau `M` contient les noms des maladies (lettres majuscules et espaces uniquement).
- Un tableau `S` contient les symptômes de chaque maladie `M[i]`, séparés par `"#"` (lettres majuscules et espaces).

On veut saisir un symptôme `Symp` et afficher toutes les maladies correspondantes.



Exemple :

Pour N = 3 :

```
M = ["ANEMIE", "TUBERCULOSE", "SPONDYLITE"]
```

```
S = ["FATIGUE#MAUX DE TETE", "TOUX#DOULEUR VERTEBRALE", "DOULEUR VERTEBRALE#FATIGUE"]
```

Pour `Symp = "DOULEUR VERTEBRALE"` → Affichage : `TUBERCULOSE`, `SPONDYLITE`.

Travail demandé :

Écrire l'algorithme du programme principal et des modules nécessaires (saisie contrôlée de `N`, `M`, `S` et recherche).

Exercice 3 : Détection de plagiat

Soient deux textes `T1` (texte d'origine) et `T2` (texte à tester) contenant le même nombre de mots. Les mots sont formés de lettres majuscules et séparés par un seul espace. `T2` est considéré comme un plagiat de `T1` si le nombre de mots de `T2` présents dans `T1` dépasse strictement la moitié du nombre total de mots de `T1`.



Exemple :

`T1 = "LE CHAT MANGE LA SOURIS"` (5 mots)

`T2 = "LA SOURIS MANGE LE FROMAGE"` (5 mots)

Mots de `T2` présents dans `T1` : `"LA"`, `"SOURIS"`, `"MANGE"`, `"LE"` (4 mots).

4 > 5 / 2 → `T2` est un plagiat de `T1`.

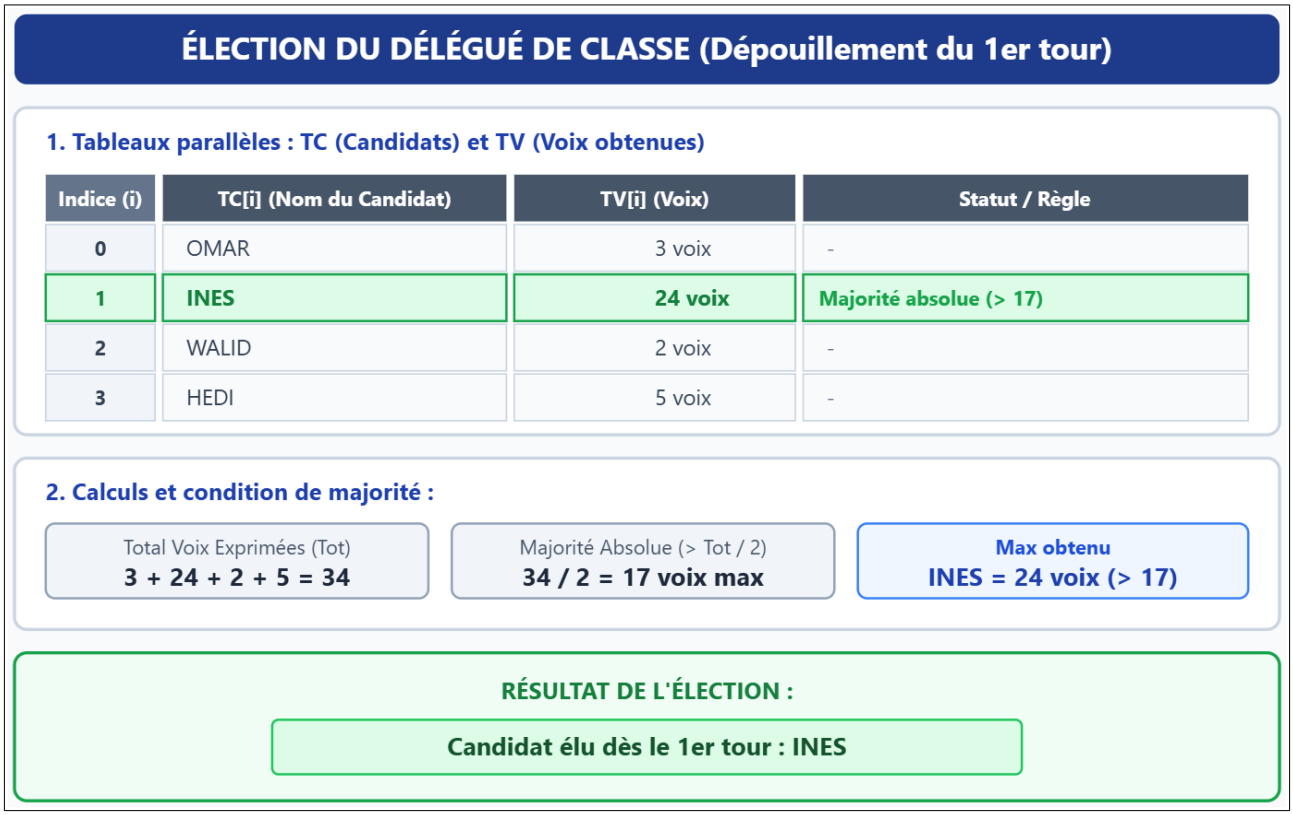
Travail demandé :

Écrire un algorithme modulaire permettant de saisir `T1` et `T2` (ayant le même nombre de mots) et de vérifier s'il y a plagiat.

Exercice 4 : Élection du délégué

Pour élire un délégué parmi N candidats ($3 \leq N \leq 9$) :

1. Remplir un tableau TC par N noms distincts (lettres majuscules uniquement).
2. Remplir un tableau TV par le nombre de voix de chaque candidat (entier aléatoire entre 0 et 40).
3. Calculer le total des voix Tot.
4. Si un candidat obtient la majorité absolue (> Tot / 2), afficher son nom. Sinon, afficher les noms des deux candidats ayant le plus de voix (par ordre décroissant).



Exemples :

Pour (N = 4) :

Cas 1 : `TC = ["OMAR", "INES", "WALID", "HEDI"]` | `TV = [3, 24, 2, 5]`

→ `Tot = 34`. Majorité > 17. Élu : `INES` (24 voix).

Cas 2 : `TC = ["OMAR", "INES", "WALID", "HEDI"]` | `TV = [12, 8, 15, 5]`

→ `Tot = 40`. Aucun candidat n'a > 20 voix. Qualifiés au 2e tour : `WALID` (15) et `OMAR` (12).

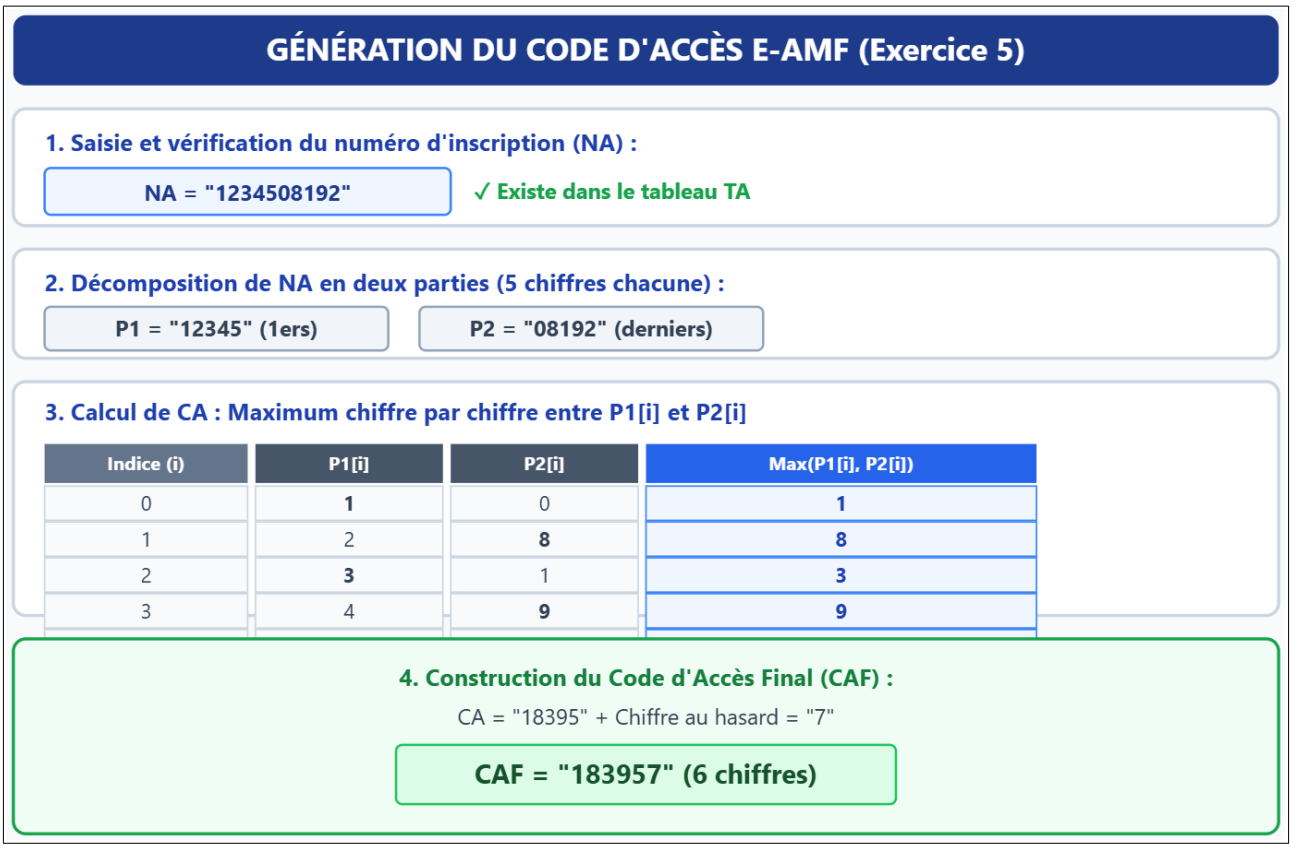
Travail demandé :

Écrire l'algorithme du programme principal et des modules envisagés.

Exercice 5 : Génération de code d'accès

Un système gère `N` assurés ($5 \leq N \leq 1000$) :

1. Remplir un tableau TA par N numéros d'inscription distincts de 10 chiffres chacun.
2. Saisir un numéro NA de 10 chiffres.
3. Si NA figure dans TA, générer un code CAF de 6 chiffres :
   - Couper NA en deux sous-chaînes de 5 chiffres : P1 (les 5 premiers) et P2 (les 5 derniers).
   - Former CA en prenant, pour chaque position i (de 0 à 4), le chiffre maximal entre P1[i] et P2[i].
   - Concaténer à la fin de CA un chiffre aléatoire entre "0" et "9".
4. Si NA ne figure pas dans TA, afficher "Numéro inexistant".



Exemple :

Pour `NA = "1234508192"` (présent dans `TA`) : `P1 = "12345"` et `P2 = "08192"`

Maximums position par position : `max(1,0)=1`, `max(2,8)=8`, `max(3,1)=3`, `max(4,9)=9`, `max(5,2)=5` → `CA = "18395"`.

Chiffre aléatoire tiré : `7` → `CAF = "183957"`.

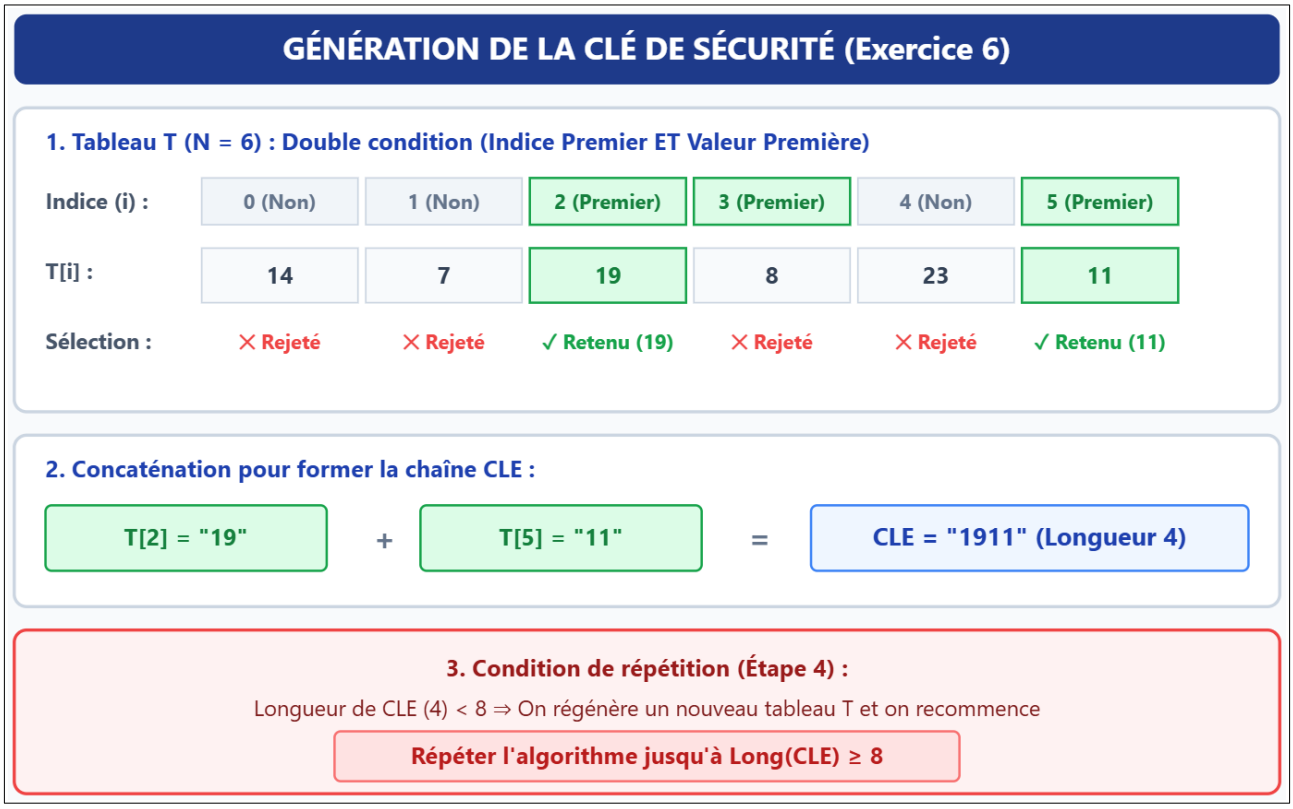
Travail demandé :

Écrire l'algorithme du programme principal et des modules envisagés.

Exercice 6 : Clé de sécurité à base de nombres premiers

On souhaite générer une clé de sécurité `CLE` d'au moins 8 chiffres selon les étapes suivantes :

1. Saisir un entier N ($10 \leq N \leq 50$).
2. Remplir un tableau T par N entiers aléatoires compris entre 2 et 100.
3. Construire la chaîne CLE en concaténant les éléments T[i] qui sont premiers ET dont l'indice i est aussi premier.
4. Répéter les étapes 2 et 3 jusqu'à ce que la longueur de CLE soit ≥ 8.



Exemple :

Pour N = 6 :

Indices `i` : `0 | 1 | 2 | 3 | 4 | 5`

Tableau `T` : `[14, 7, 19, 8, 23, 11]`

Indices premiers : 2, 3, 5.

`i = 2` : `T[2] = 19` (premier → retenu)

`i = 3` : `T[3] = 8` (non premier → ignoré)

`i = 5` : `T[5] = 11` (premier → retenu)

`CLE = "1911"` (longueur 4 < 8, donc on régénère un nouveau tableau `T`).

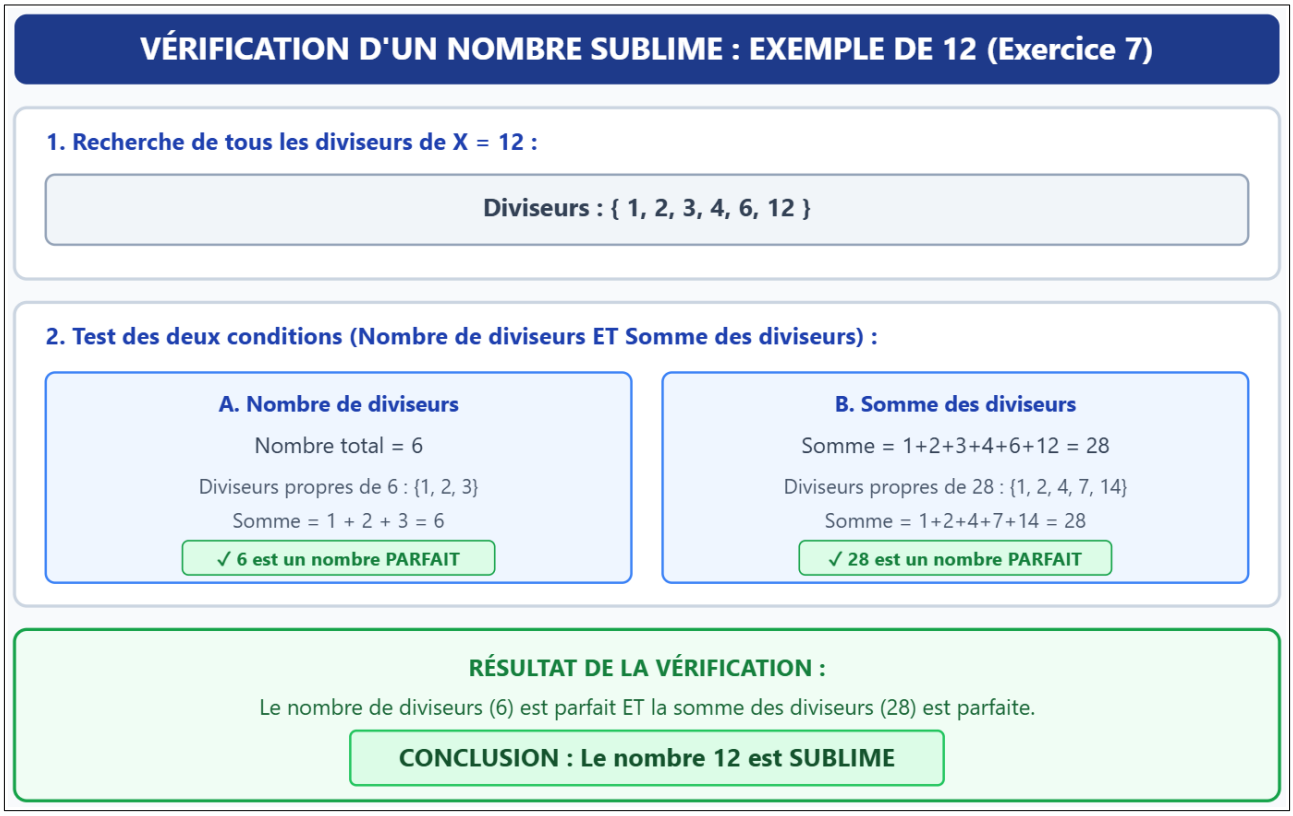
Travail demandé :

Écrire l'algorithme du programme principal et des modules envisagés.

Exercice 7 : Les nombres sublimes

Un entier est **parfait** s'il est égal à la somme de ses diviseurs propres (strictement inférieurs à lui-même).

Un entier est **sublime** si le nombre de tous ses diviseurs et la somme de tous ses diviseurs sont deux nombres parfaits.



Exemple :

Pour `X = 12` :

Diviseurs de 12 : `1, 2, 3, 4, 6, 12`.

Nombre de diviseurs = `6` (qui est parfait car 1 + 2 + 3 = 6).

Somme des diviseurs = `28` (qui est parfait car 1 + 2 + 4 + 7 + 14 = 28).

Donc `12` est un nombre sublime.

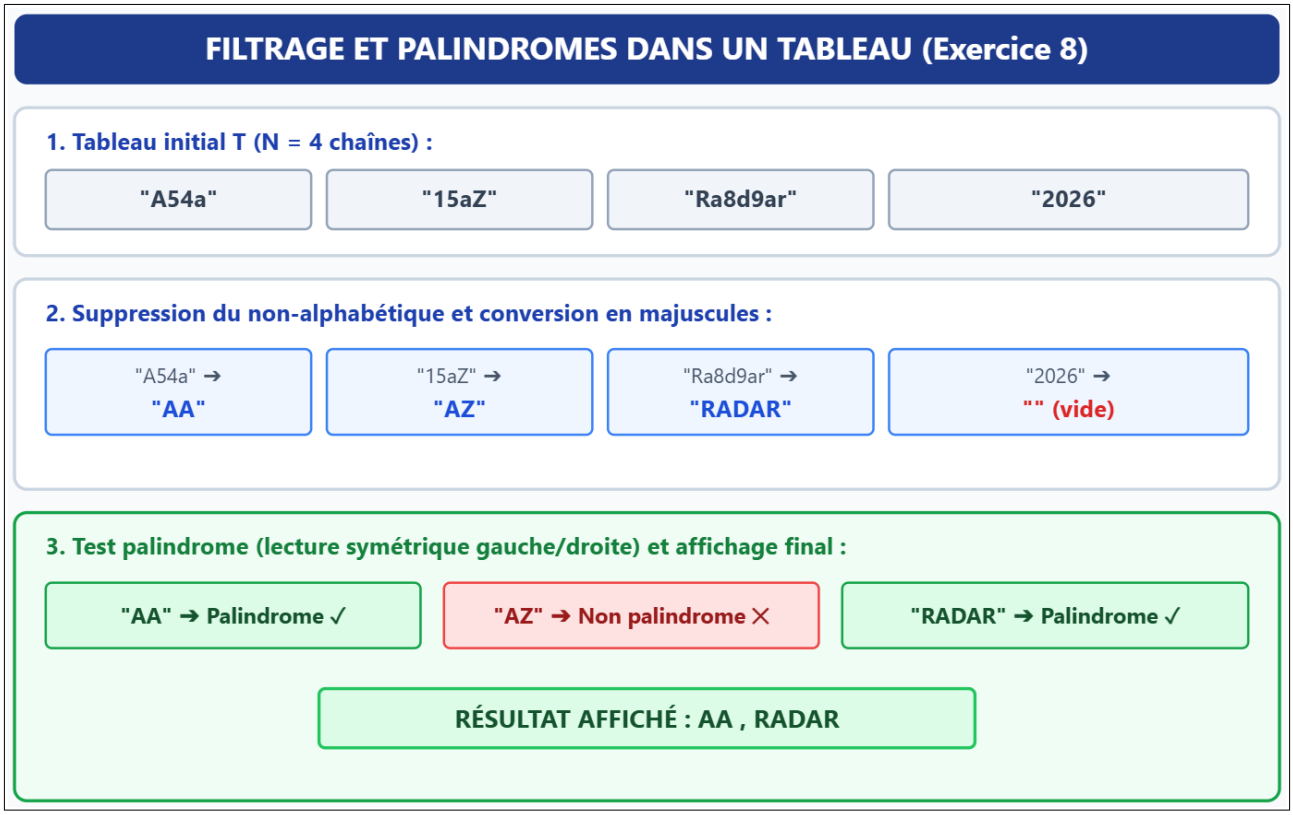
Travail demandé :

Écrire un algorithme modulaire qui remplit un tableau `T` par `N` entiers ($5 \leq N \leq 30$, avec $T[i] \geq 10$) et affiche tous les nombres sublimes de `T`.

Exercice 8 : Filtrage et Palindromes

Soit un tableau `T` de `N` chaînes non vides ($2 \leq N \leq 30$, longueur maximale de chaque chaîne = 10). Pour chaque chaîne `T[i]` :

1. Supprimer tous les caractères non alphabétiques.
2. Convertir les lettres restantes en majuscules.
3. Afficher la chaîne obtenue si elle est non vide et palindrome (se lit de la même façon dans les deux sens).



Exemple :

Pour N = 4 :

`T` initial : `["A54a", "15aZ", "Ra8d9ar", "2026"]`

Après nettoyage et mise en majuscules : `["AA", "AZ", "RADAR", ""]`

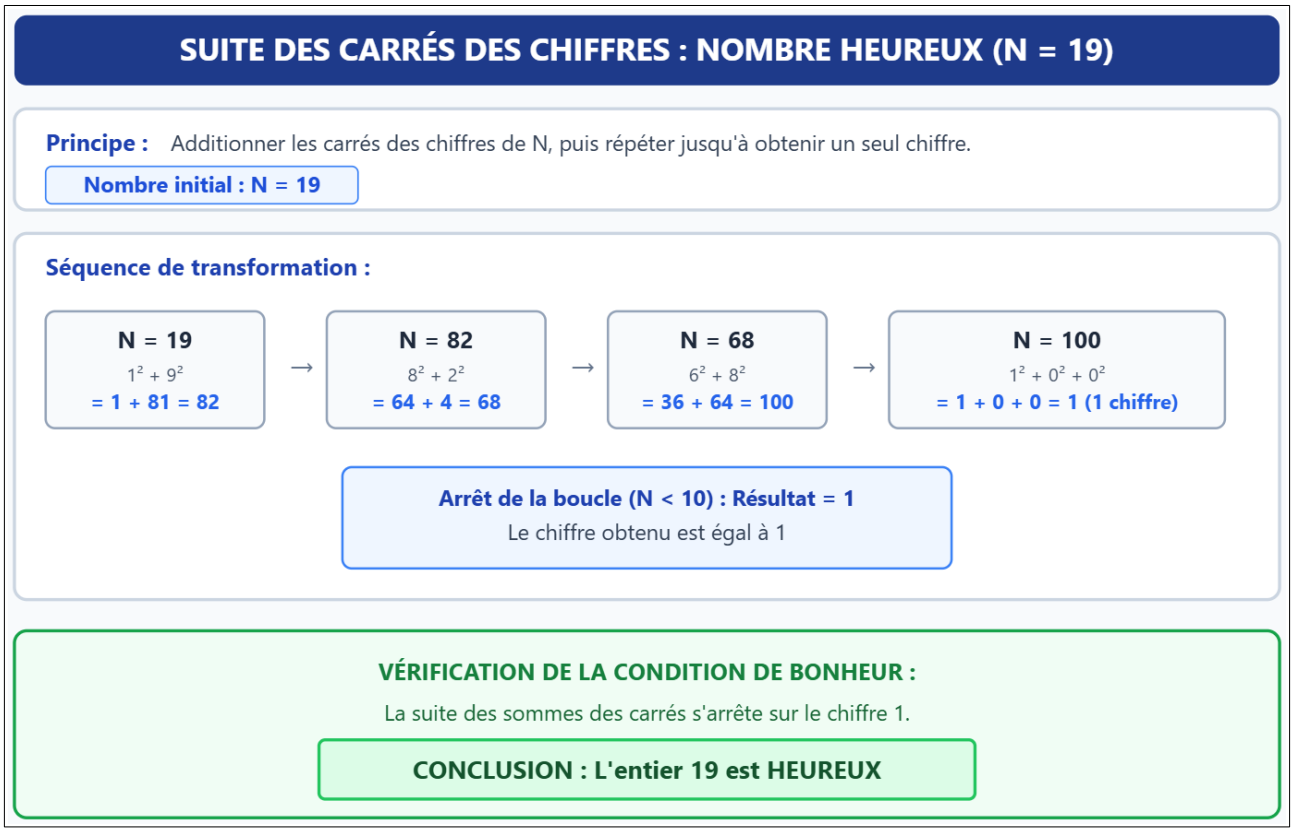
Palindromes affichés : AA, RADAR.

Travail demandé :

Écrire l'algorithme du programme principal et des modules nécessaires.

Exercice 9 : Les nombres heureux

Pour savoir si un entier N ≥ 10 est heureux, on remplace `N` par la somme des carrés de ses chiffres, et on répète l'opération jusqu'à obtenir un nombre à un seul chiffre (< 10). Si ce chiffre final est `1`, le nombre est heureux.



Exemples :

`N = 19` → 1^2 + 9^2 = 82 → 8^2 + 2^2 = 68 → 6^2 + 8^2 = 100 → 1^2 + 0^2 + 0^2 = 1 (`19` est heureux).

`N = 20` → 2^2 + 0^2 = 4 (chiffre < 10 différent de 1 → `20` n'est pas heureux).

Travail demandé :

1. Écrire l'algorithme de la fonction SommeCarre(X: Entier): Entier qui retourne la somme des carrés des chiffres de X.
2. Écrire l'algorithme de la fonction Heureux(N: Entier): Booléen qui vérifie si N est heureux en exploitant SommeCarre.

Exercice 10 : Maîtrise des sous-programmes

Partie A : Validité des appels

Soient les variables globales : `S, M` (Entier), `T` (Tableau de 40 entiers), `B` (Booléen), `R` (Réel).

Soient les entêtes suivants :

- `Procédure Traiter (@N: Entier)`
- `Fonction Tester (Ch: Chaîne): Booléen`
- `Fonction Calculer (T: Tab; N: Entier): Réel`
- `Procédure Chercher (@X: Réel; N: Entier)`

Indiquer si chaque appel est Valide ou Invalide et justifier :

1. `B ← Tester("Bac")`
2. `Ecrire(Calculer(T, 6))`
3. `Traiter(S, B)`
4. `Chercher(M, T)`
5. `Traiter(S + 1)`
6. `M ← Calculer(T, S)`

Partie B : Déduction d'entêtes

Donner l'entête complet de chaque fonction à partir de son appel (`X` est un entier) :

1. `Y ← Verifier(X)` suivi de `Si Y = Vrai Alors ...`
2. `Si Trouver("A", "BAC") = -1 Alors ...`
3. `Z ← Traiter(X / 2) + "#"`

Partie C : Correction d'erreurs

Identifier l'erreur syntaxique ou logique dans chacune des déclarations suivantes :

1. Une fonction `PGCD(a, b: Entier): Entier` qui calcule le PGCD dans `a` mais ne contient pas l'instruction `Retourner`.
2. Une fonction `Fact(N: Entier): Chaîne` qui calcule la factorielle dans un entier `F` et exécute `Retourner F`.
3. Une fonction `Premier(N: Entier): Booléen` qui compte le nombre de diviseurs dans un entier `Nb` et exécute `Retourner Nb`.